
Document Méthodologie et Organisation du Projet

Jalon 2 – Projet Fil Rouge CDA

Étudiant FORTUNATO Axel

Formation IPSSI — CDA Bachelor Fullstack DevOps

Promotion 2025–2026

Date Février 2026

Table des matières

1. Méthode de gestion de projet	2
1.1 Méthodologie : Kanban adapté	2
1.2 Organisation	2
2. Planning global	3
2.1 Jalons	3
2.2 Planning détaillé	3
2.3 Tâches critiques	4
3. Outils de suivi	5
3.1 GitHub Projects (Kanban)	5
3.2 Milestones	7
3.3 Mise à jour	7
4. Gestion code source (Git)	8
4.1 Dépôt	8
4.2 Stratégie branches (Git Flow simplifié)	8
4.3 Workflow de développement	8
4.4 Convention de commits	9
5. CI/CD planifié	10
5.1 Intégration Continue (CI)	10
5.2 Déploiement Continu (CD)	10
Conclusion	10

1. Méthode de gestion de projet

1.1 Méthodologie : Kanban adapté

J'ai choisi une **approche Kanban simplifiée** pour ce projet solo.

Justification : - **Flexibilité** : Pas besoin de sprints rigides en travaillant seul - **Adaptabilité** : Ajustement continu des priorités - **Visibilité** : Vue claire de l'avancement - **Cohérence** : Les 6 jalons sont des points de validation naturels

1.2 Organisation

- Tâches de 2-4 heures
- Max 3 tâches simultanées
- Revue hebdomadaire (dimanche)
- Auto-revue du code avant merge

Note linguistique :

Conformément aux bonnes pratiques de développement, le code source, les commits Git, les issues GitHub et la documentation technique sont rédigés en **anglais** (langue internationale du développement).

En revanche, les documents de gestion de projet (CDCF, méthodologie, rapports) et l'interface utilisateur de l'application sont en **français** (langue du public cible et du jury).

2. Planning global

2.1 Jalons

Jalon	Mois	Date	Livrable
1	Jan	31/01	CDCF
2	Fév	28/02	Méthodologie + UI/UX
3	Mar	31/03	Modélisation BDD
4	Avr	30/04	Conception + Début dev
5	Mai	30/05	Bêta + Tests + Sécurité
6	Juin	30/06	Version finale

2.2 Planning détaillé

FÉVRIER 2026

- Sem 1-2 : Setup (Git, planif CI/CD)
- Sem 3-4 : Conception UI/UX complète

MARS 2026

- Sem 1 : Dictionnaire données + Entités
- Sem 2 : MCD (Modèle Conceptuel)
- Sem 3 : MLD + MPD
- Sem 4 : Scripts SQL + jeu de test

AVRIL 2026

- Sem 1 : Diagrammes UML (cas utilisation, séquence)
- Sem 2 : Diagramme classes + Architecture
- Sem 3-4 : Dev backend (Symfony, Docker, Auth, CRUD)

MAI 2026

- Sem 1-2 : API (lecture, pagination, favoris, text-to-speech)
- Sem 2-3 : Frontend React (composants, pages, API)
- Sem 3-4 : Tests (unit, fonct) + Sécurité + CI

JUIN 2026

- Sem 1 : Finalisations + bugs
- Sem 2 : Déploiement Docker
- Sem 3 : Documentation finale
- Sem 4 : Présentation soutenance

2.3 Tâches critiques

- Validation modélisation BDD (fin mars)
- Docker opérationnel (mi-avril)
- API auth fonctionnelle (fin avril)
- Intégration text-to-speech (début mai)
- Tests automatisés (mi-mai)

20% du temps réservé aux imprévus chaque mois.

3. Outils de suivi

3.1 GitHub Projects (Kanban)

Outil choisi : GitHub Projects intégré au dépôt

Avantages : - Gratuit et intégré à GitHub - Centralise code et gestion - Professionnel

Organisation :

Le board Kanban utilise 6 colonnes :

Colonne	Signification
[Backlog]	Tâches en attente
[To Do]	À faire cette semaine
[In Progress]	En cours (max 3)
[Done]	Terminé
[Bugs]	Anomalies à corriger
[Documentation]	Rédaction

Code couleur (convention professionnelle) : - Backlog → Gris (neutre) - To Do → Bleu (planifié) - In Progress → Jaune (actif) - Done → Vert (succès) - Bugs → Rouge (urgent) - Documentation → Violet (support)

Le board Kanban suivant illustre l'organisation du projet en février 2026 :

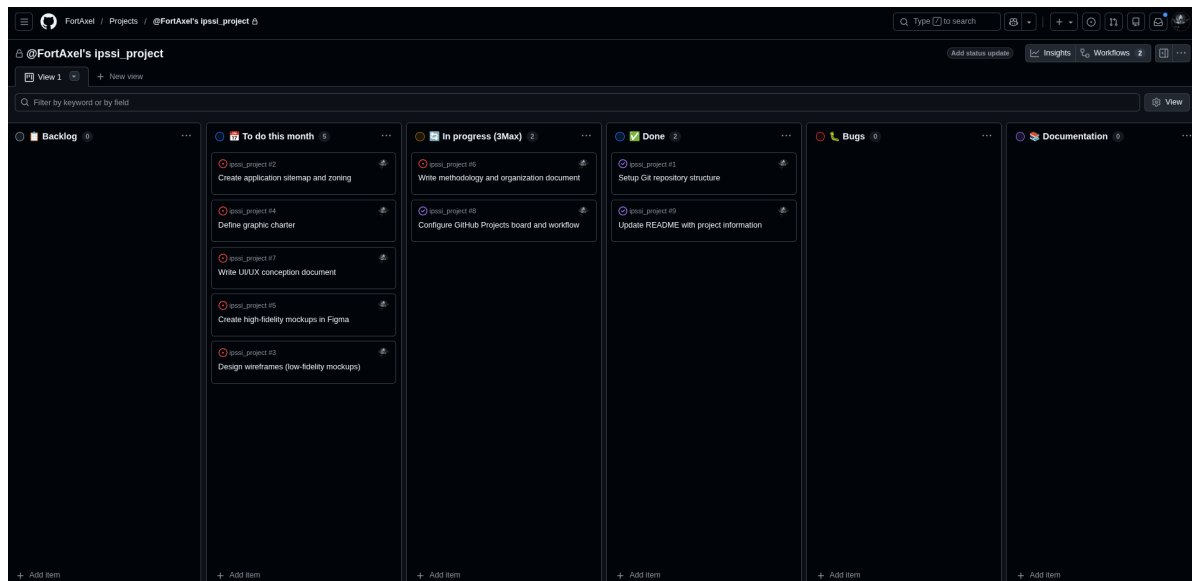


Fig. 1 : GitHub Projects Board

Chaque issue GitHub contient une description structurée, des labels, et une assignation au milestone correspondant :

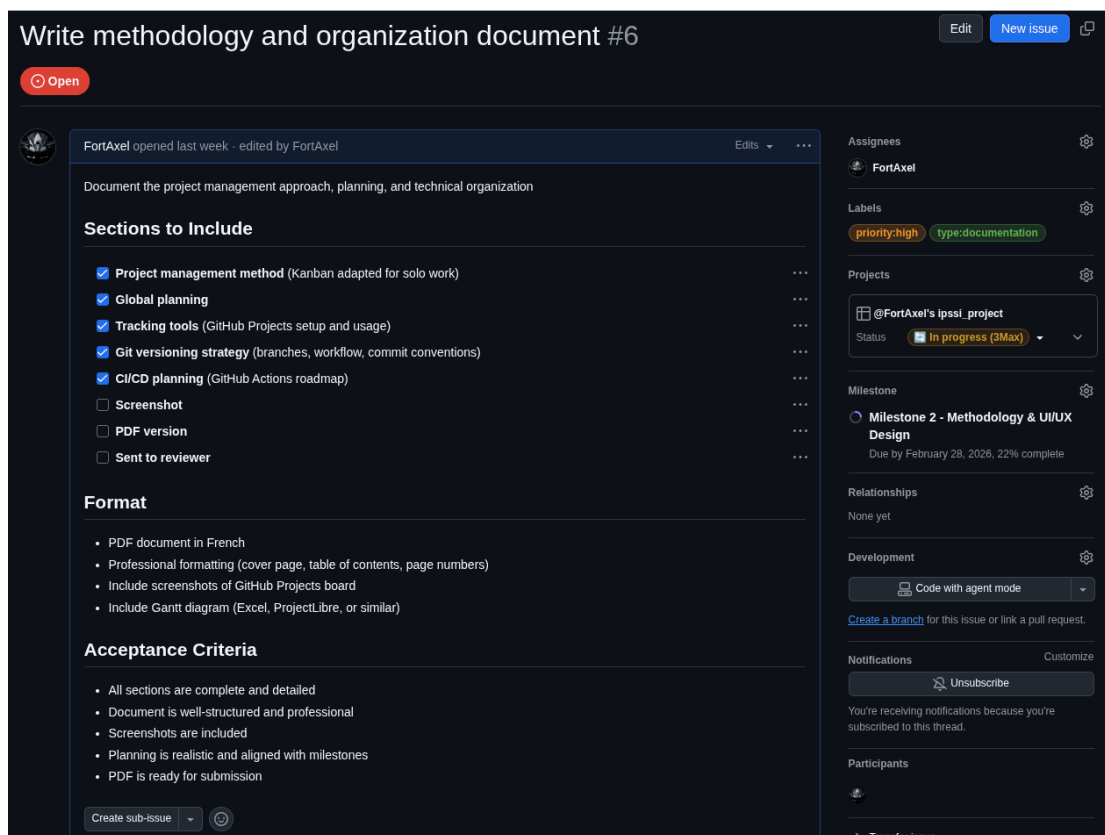


Fig. 2 : Issue détaillée

3.2 Milestones

Chaque jalon est représenté par un Milestone GitHub : - Milestone 1 : CDCF (fait) - Milestone 2 : Méthodologie + UI/UX - Milestone 3 : Modélisation BDD - Milestone 4 : Conception + Dev - Milestone 5 : Bêta + Tests - Milestone 6 : Version finale

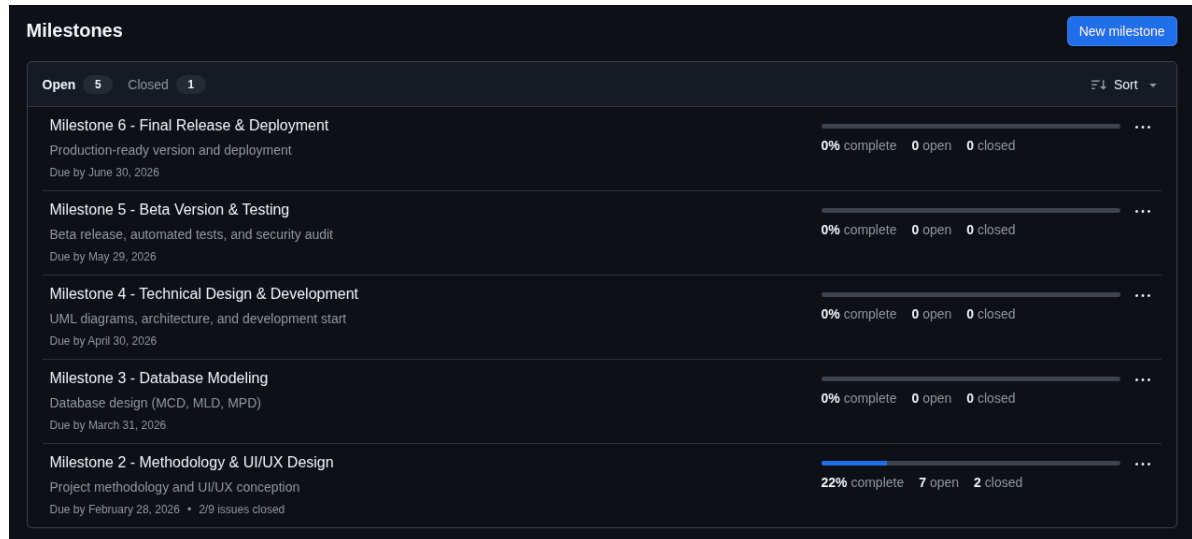


Fig. 3 : Milestones

3.3 Mise à jour

- **Quotidien** : Déplacement des tâches dans les colonnes
- **Hebdomadaire** : Revue et planification (dimanche)
- **Mensuel** : Bilan du jalon

4. Gestion code source (Git)

4.1 Dépôt

- **Plateforme** : GitHub
- **URL** : https://github.com/FortAxel/ipssi_project
- **Visibilité** : Privé (accès formateur)

4.2 Stratégie branches (Git Flow simplifié)

```
main (production stable)
|
└─ develop (intégration)
    |
    ├── feature/user-auth
    ├── feature/story-catalog
    ├── feature/story-reading
    ├── feature/favorites
    ├── feature/admin-panel
    └─ feature/text-to-speech
```

Branches : - main : versions stables jalons (protégée, taggée) - develop : intégration continue (toujours fonctionnelle) - feature/* : une par fonctionnalité majeure - hotfix/* : corrections urgentes

4.3 Workflow de développement

1. Créer une branche feature depuis develop
`git checkout develop`
`git checkout -b feature/story-catalog`
2. Développer et commiter régulièrement
`git add .`
`git commit -m "feat: add story list endpoint"`
3. Merger dans develop une fois terminé
`git checkout develop`

```
git merge feature/story-catalog

4. Supprimer la branche feature
git branch -d feature/story-catalog

5. Push sur GitHub
git push origin develop
```

Note sur les Pull Requests :

Bien que les Pull Requests soient une bonne pratique en équipe pour la revue de code collaborative, elles ne sont pas utilisées dans ce projet solo pour éviter une charge administrative inutile. En revanche, une **auto-revue systématique** du code est effectuée avant chaque merge (relecture du diff, vérification des tests, cohérence avec l'architecture).

4.4 Convention de commits

Format : <type>: <description>

Types : - feat : nouvelle fonctionnalité - fix : correction bug - docs : documentation - refactor : refactorisation

Exemples :

```
feat: add user login endpoint
fix: correct pagination offset calculation
docs: update installation instructions
test: add unit tests for story service
refactor: improve page navigation logic
```

Note : Les messages de commit sont en anglais (convention internationale), tandis que les documents projet et l'interface utilisateur sont en français.

5. CI/CD planifié

5.1 Intégration Continue (CI)

Outil : GitHub Actions (gratuit, intégré)

Objectif : Automatiser les tests à chaque modification du code

Pipeline prévue (mise en place en avril-mai) : 1. Vérification du code (PHP CS Fixer) 2. Tests unitaires (PHPUnit) 3. Build Docker (validation)

Déclencheur : Push sur develop et main

Résultat : Tests OK → code validé | Tests KO → corrections nécessaires

5.2 Déploiement Continu (CD)

Objectif : Automatiser le déploiement (mise en place en juin)

Actions prévues : - Build de l'image Docker finale - Push sur Docker Hub - Tagging des versions (v1.0, v2.0...)

Mise en œuvre : Progressive à partir d'avril (Jalon 4).

Conclusion

L'approche Kanban + Git Flow + CI/CD progressive garantit qualité et organisation. Le planning sera ajusté selon l'avancement réel et retours formateur.

Prochaine étape : Conception UI/UX (partie 2 Jalon 2)

Version : 1.0

Date : Février 2026